ComfortSense: A Smart Embedded System for Occupancy, Comfort, and Energy Monitoring

By Alexander Samuel R

Introduction

Picture this: an exam hall, an interview waiting room, a boardroom with a client flying in from abroad. The room is quiet, your palms aren't. Cortisol climbs, the air feels thick and warm, and your body starts looking for a way out before your mind even catches up.

This is a problem nobody designs around. Research in indoor environmental quality consistently shows that elevated temperature and humidity measurably degrade cognitive performance: concentration, memory, and decision-making all decline before a person ever consciously registers discomfort. Exam halls, interview lounges, and meeting rooms hosting international delegates are already high-stakes by nature; poor air conditions quietly stack the odds against the people inside them.

ComfortSense is a smart embedded system built to catch this before it becomes a problem. It continuously tracks indoor and outdoor temperature, humidity, and real-time occupancy, converting raw sensor data into a live comfort score for the room. But it doesn't stop at monitoring. The same sensor fusion doubles as an unobtrusive security layer, flagging motion when the room should be empty, and an energy-saving system that automatically dims lighting based on occupancy and ambient brightness, a small, direct contribution toward UN Sustainable Development Goal 7: Affordable and Clean Energy.

Built around a single NodeMCU ESP8266, a handful of environmental sensors, and a lean piece of firmware logic, ComfortSense makes a simple case: the room someone walks into right before the most important ten minutes of their life shouldn't be working against them.

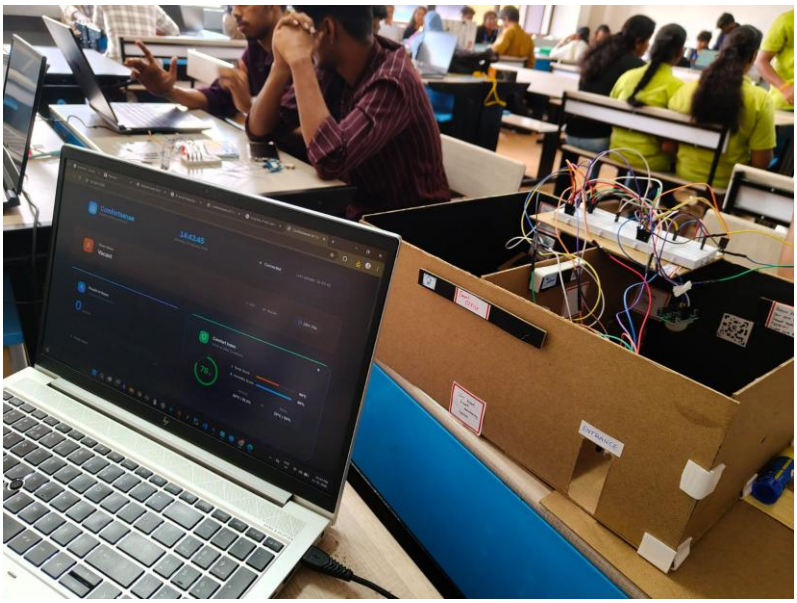

Fig.1: Complete Comfort Sense Setup

Components Required

Component	Quantity
ESP8266 NodeMCU	1
Breadboards	2
DHT22 Temperature & Humidity Sensor	2
PIR Motion Sensor	1
IR Obstacle Sensor	2
LDR (Photoresistor)	1
White LED	1
Active Buzzer	1
220 Ω Resistor	1
10 kΩ Resistor	1
Male-Male Jumper Wires	~20–30
Male-Female Jumper Wires	As required
USB Cable (Micro-USB)	1
5V Power Bank / USB Power Supply	1 (optional)
Laptop (Flask Server)	1

Circuit and Working

ESP8266 Pin	Component	Purpose
D0 (GPIO16)	White LED (+220Ω resistor)	Smart room lighting
D1 (GPIO5)	IR Sensor 1	Entry detection
D2 (GPIO4)	DHT22 (Indoor)	Indoor temperature & humidity
D4 (GPIO2)	DHT22 (Outdoor)	Outdoor temperature & humidity
D5 (GPIO14)	Active Buzzer	High humidity alarm
D6 (GPIO12)	IR Sensor 2	Exit detection
D7 (GPIO13)	PIR Motion Sensor	Motion detection
A0	LDR + 10kΩ resistor	Ambient light sensing

ESP8266 Pin	Component	Purpose
3.3V	DHT22s, PIR, IR sensors, LDR circuit	Sensor power
GND	All sensor grounds	Common ground

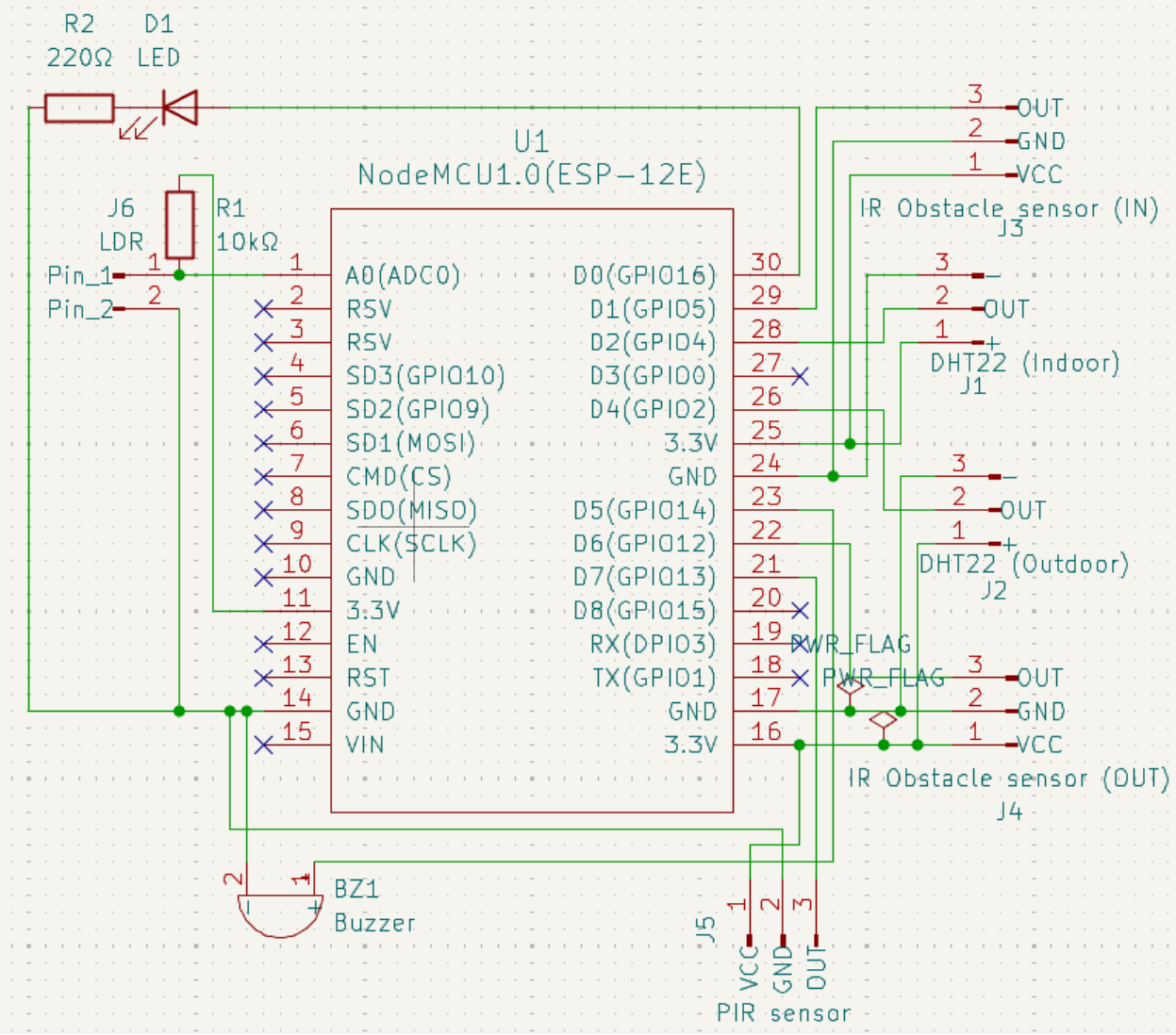

Fig.2: Schematic of Comfort Sense

Comfort Monitoring: Dual DHT22 Setup

Two DHT22 sensors work in tandem to read both sides of the comfort equation. The indoor unit continuously tracks temperature and humidity inside the room; cross 30°C or 70% relative humidity and the dashboard status immediately flips to critical, with the audible buzzer reserved specifically for sustained high humidity conditions. The outdoor unit reads ambient conditions just outside the door and is compared against the indoor reading to calculate the thermal delta, a live measure of how hard, and how effectively, the room's ventilation or AC system is working. The system doesn't just report that a room is uncomfortable; it tells the manager why.

People Counting: Dual IR Sensor Logic

Two IR sensors are mounted roughly 3cm apart above the doorway. If the entry sensor trips first and the exit sensor follows within a one second window, the firmware registers one person entering; the reverse sequence, exit first, then entry, registers an exit. This sequential trigger logic, rather than reading either sensor in isolation, is what lets a single doorway distinguish direction instead of just detecting that something passed through. The result is a live, accurate headcount the manager can use to catch overcrowding before it happens.

Automatic Lighting: LDR + IR Integration

When the IR-derived headcount is above zero and the LDR reads low ambient light, the system brightens the LED proportionally, enough to meet comfort levels without wasteful over-illumination. No people, no light. It's a small rule with a real effect: lighting that tracks actual room use instead of running on a fixed schedule.

Security: PIR Motion Detection

A PIR sensor inside the room runs continuously alongside the IR-based headcount. If PIR registers motion while the people count reads zero, the only explanation is someone entered without passing through the monitored doorway, a clear intrusion signal, flagged immediately to the manager and security staff.

The Buzzer: Offline Alert

A buzzer sits on the manager's desk rather than the monitored room itself. It sounds specifically when indoor humidity crosses 70% while the room is occupied, the one condition the firmware ties to an audible alert, with a built-in 10 second cooldown so it warns without becoming a continuous nuisance. The manager stays informed without needing to keep a dashboard open all day.

Software

Toolchain and Stack

- Firmware: Arduino IDE, Adafruit's DHT library, ESP8266WiFi, ESP8266HTTPClient
- Backend: Python, Flask, Flask-CORS, SQLite for persistent storage
- Frontend: HTML, CSS, JavaScript, Chart.js for the live sensor-history graph
- Communication: JSON over HTTP POST, sent from the NodeMCU to the Flask server once every second

ComfortSense_Compact §

```
#include <DHT.h>
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

const char* SSID       = "your wifi ssid";
const char* PASSWORD   = "your password";
const char* SERVER_URL = "http://10.165.141.186:5005/api/update";

#define DHT_IN   D4
#define DHT_OUT  D2
#define PIR      D7
#define BUZZER   D5
#define LDR      A0
#define LED      D0
#define IR_ENTRY D1
#define IR_EXIT  D6

DHT dhtIn(DHT_IN, DHT22), dhtOut(DHT_OUT, DHT22);

#define TEMP_MAX     30.0f
#define HUM_MAX      70.0f
#define LDR_DARK     150
#define IDLE_TIMEOUT 60000UL
#define T_DHT        2000UL
#define T_SEND       1000UL
#define T_BUZZ       10000UL
#define T_IR_WIN     1000UL

bool  roomActive = false, buzzerMuted = false;
int   peopleCount = 0, ldrValue = 0;
int   brightness = 0, targetBrightness = 0;
float tIn = 0, hIn = 0, tOut = 0, hOut = 0;
unsigned long tLastMotion = 0, tLastBuzz = 0, tLastSend = 0, tLastDHT = 0, tIRSeq = 0;

enum IRState { IDLE, WAIT_IR2, WAIT_IR1 } irState = IDLE;
WiFiClient wifiClient;
HTTPClient http;

void beep() { digitalWrite(BUZZER, HIGH); delay(120); digitalWrite(BUZZER, LOW); }
```

ComfortSense_Compact §

```
void setup() {
  Serial.begin(115200);
  pinMode(PIR, INPUT); pinMode(IR_ENTRY, INPUT); pinMode(IR_EXIT, INPUT);
  pinMode(BUZZER, OUTPUT); pinMode(LED, OUTPUT);
  digitalWrite(BUZZER, LOW); digitalWrite(LED, LOW);
  dhtIn.begin(); dhtOut.begin();

  WiFi.begin(SSID, PASSWORD);
  for (int i = 0; WiFi.status() != WL_CONNECTED && i < 20; i++) delay(500);
  Serial.println(WiFi.status() == WL_CONNECTED ? "WiFi connected" : "Offline mode");
}

void loop() {
  unsigned long now = millis();

  // LDR -> adaptive LED brightness
  ldrValue = 0;
  for (int i = 0; i < 10; i++) { ldrValue += analogRead(LDR); delay(3); }
  ldrValue /= 10;
  targetBrightness = (peopleCount > 0 && ldrValue < LDR_DARK)
    ? constrain(map(ldrValue, 0, LDR_DARK, 255, 0), 0, 255) : 0;
  brightness += (brightness < targetBrightness) ? 5 : (brightness > targetBrightness) ? -5 : 0;
  brightness = constrain(brightness, 0, 255);
  analogWrite(LED, brightness);

  // Dual IR -> directional people count
  bool e = (digitalRead(IR_ENTRY) == LOW), x = (digitalRead(IR_EXIT) == LOW);
  switch (irState) {
    case IDLE:
      if (e) { irState = WAIT_IR2; tIRSeq = now; }
      else if (x) { irState = WAIT_IR1; tIRSeq = now; }
      break;
    case WAIT_IR2:
      if (x) { peopleCount++; irState = IDLE; delay(250); }
      else if (now - tIRSeq > T_IR_WIN) irState = IDLE;
      break;
    case WAIT_IR1:
      if (e) { if (peopleCount > 0) peopleCount--; irState = IDLE; delay(250); }
      else if (now - tIRSeq > T_IR_WIN) irState = IDLE;
      break;
```

ComfortSense_Compact §

```
    }

    // PIR -> occupancy with idle timeout
    if (digitalRead(PIR) == HIGH) { tLastMotion = now; roomActive = true; }
    if (peopleCount == 0 && now - tLastMotion > IDLE_TIMEOUT) roomActive = false;

    // DHT22 read + humidity buzzer alert
    if (now - tLastDHT >= T_DHT) {
      float ti = dhtIn.readTemperature(),  hi = dhtIn.readHumidity();
      float to = dhtOut.readTemperature(), ho = dhtOut.readHumidity();
      if (!isnan(ti)) { tIn = ti; hIn = hi; tOut = to; hOut = ho; }
      if (hIn >= HUM_MAX && !buzzerMuted && roomActive && now - tLastBuzz >= T_BUZZ) {
        beep(); tLastBuzz = now;
      }
      if (hIn < HUM_MAX) buzzerMuted = false;
      tLastDHT = now;
    }

    // POST JSON to Flask every second
    if (now - tLastSend >= T_SEND && WiFi.status() == WL_CONNECTED) {
      String payload = "{\"indoor_temp\":"  + String(tIn, 1) +
                       ",\"indoor_hum\":"   + String(hIn, 1) +
                       ",\"outdoor_temp\":" + String(tOut, 1) +
                       ",\"outdoor_hum\":"  + String(hOut, 1) +
                       ",\"light\":"        + ldrValue +
                       ",\"brightness\":"   + map(brightness, 0, 255, 0, 100) +
                       ",\"motion\":"       + (roomActive ? 1 : 0) +
                       ",\"people\":"       + peopleCount +
                       ",\"occupancy\":\""  + (roomActive ? "occupied" : "vacant") + "\"}";
      http.begin(wifiClient, SERVER_URL);
      http.addHeader("Content-Type", "application/json");
      http.POST(payload);
      http.end();
      tLastSend = now;
    }

    delay(10);
}
```

Fig.3: Screenshots of Firmware code

Firmware Logic

The IR-based people counter runs as a small state machine rather than reading either sensor in isolation. The firmware waits for the second sensor to trip within a defined window after the first, which is what lets it tell entry apart from exit instead of just registering that something passed. The LED doesn't snap to full brightness either; it eases up or down a few steps every loop, giving a smooth transition instead of a jarring on/off flick. All of this runs on non-blocking timers, so the IR check, PIR check, DHT reads, and the WiFi POST to Flask can all happen in the same loop without one waiting on another.

The Dashboard: Full-Stack Visibility

The web application ingests a fresh reading every second and turns it into a live operations view: temperature, humidity, and outdoor comparisons; real-time people count and crowd status; AC and ventilation efficiency; security alerts; an energy cost meter calculated at the standard Indian

tariff of ₹6.5/kWh; and a live graph of temperature, humidity, and outdoor readings, with a CSV export for the full historical record.

A couple of resilience touches sit underneath that view: the backend flags a sensor as stale if its reading hasn't changed across several consecutive updates, and falls back to a gradual occupancy-based estimate for people count if the hardware ever fails to report one explicitly, so the dashboard doesn't silently go blind if one signal drops out.

Full source for both the firmware and the Flask backend is available at https://github.com/alexander-devstack/Comfort-Sense

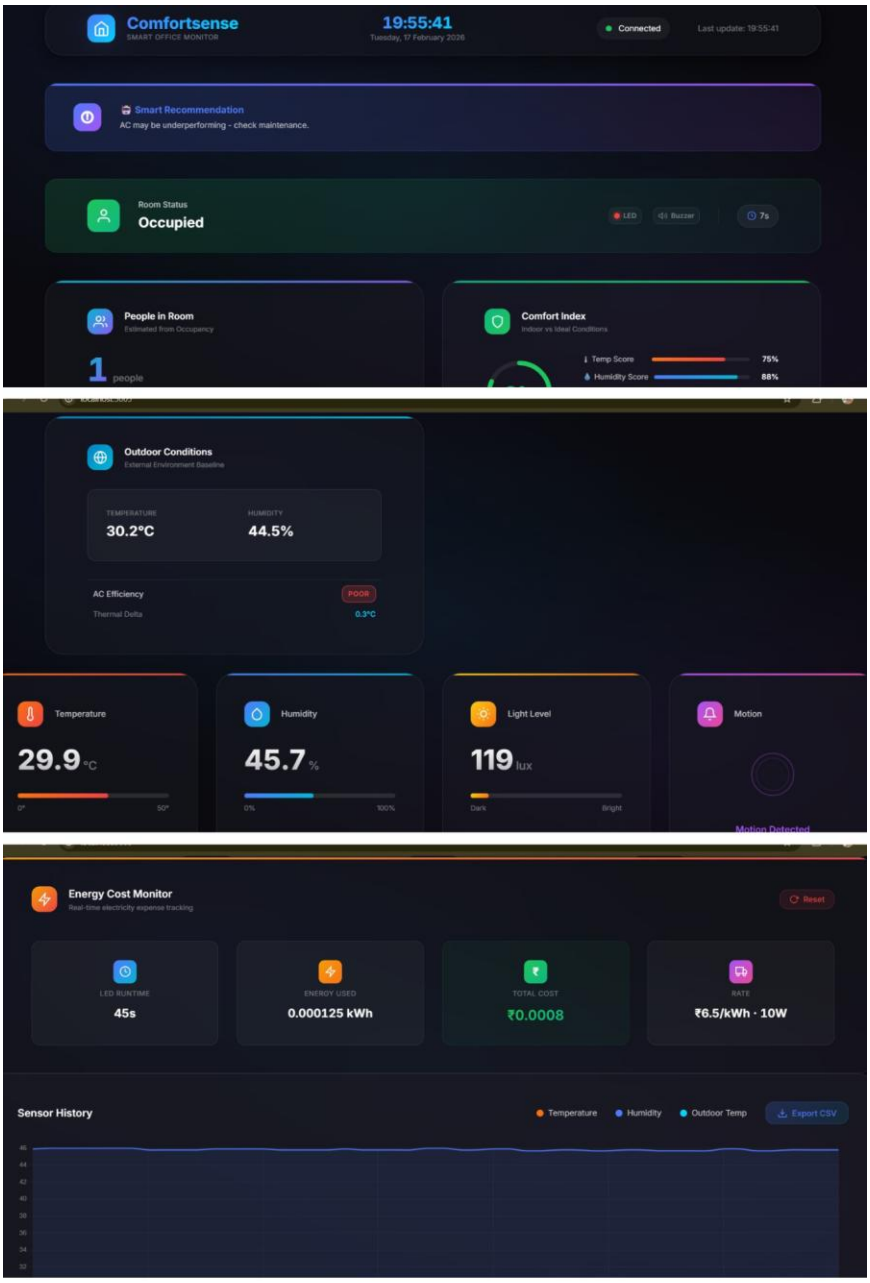

Fig.4: Screenshots of Comfort Sense's Dashboard

Construction and Testing

Assembly and Deployment

ComfortSense was deployed across two adjoining spaces in a real office layout: a manager's room and an interviewee waiting room. The waiting room houses the bulk of the sensing hardware, the dual IR pair and PIR sensor at the doorway, the indoor DHT22, and the LDR-driven LED, while the manager's room holds just the buzzer and the laptop running the dashboard, kept accessible without requiring the manager's constant attention. A second DHT22 is mounted externally, outside the waiting room, purely as an outdoor reference for the comfort comparison. Physical wiring follows the pin table in the Circuit and Working section directly; no additional assembly steps beyond connecting each sensor to its assigned GPIO.

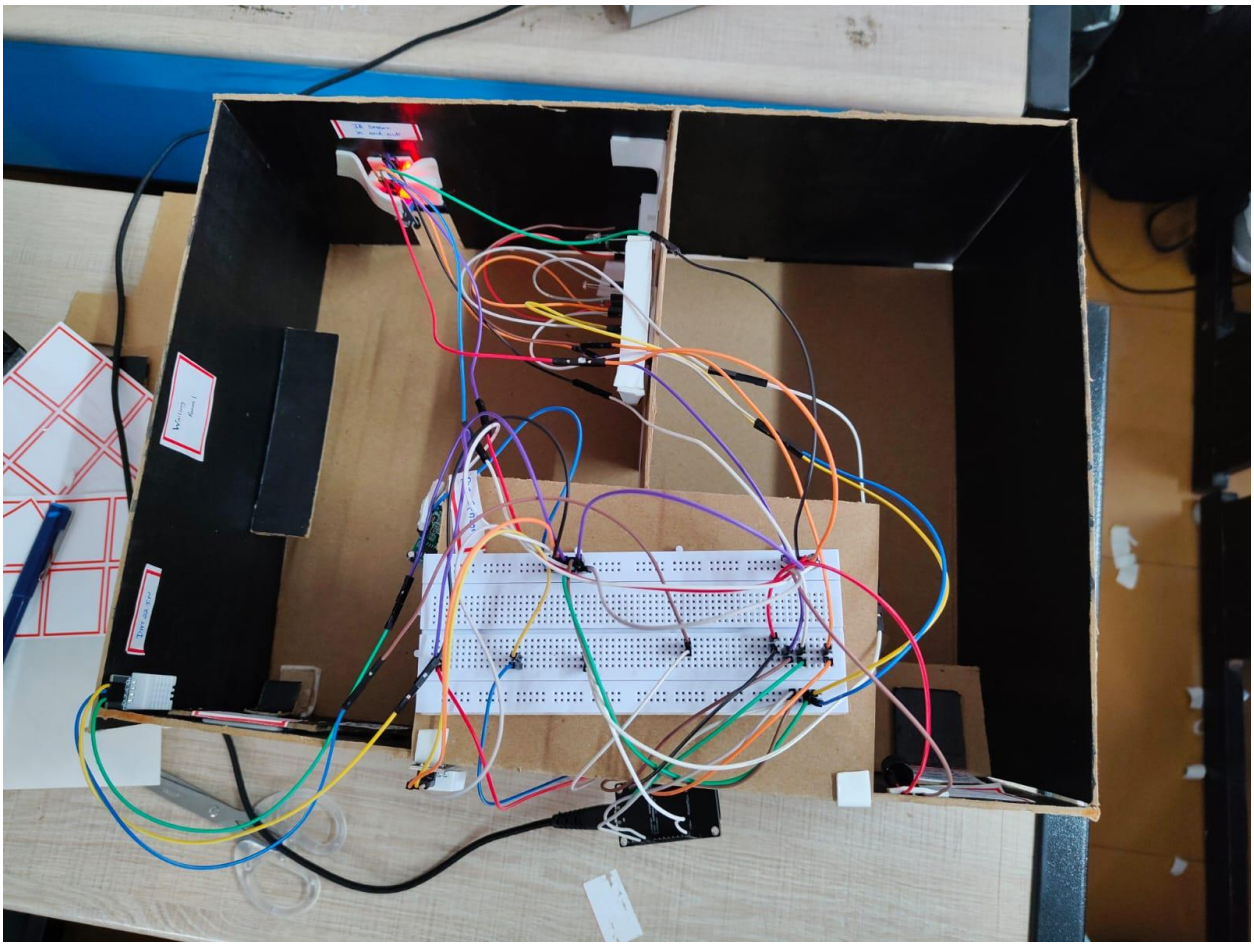

Fig.5: Top view of Comfort Sense acting in an open space

Calibration

The LDR threshold (LDR_DARK = 150 in firmware) is a fixed value tuned manually for the waiting room's ambient lighting. There is no adaptive or machine-learning component deciding this on its own, so the threshold needs re-tuning by hand if the system is redeployed in a space with different lighting conditions.

The dual DHT22 comparison does double duty here: beyond flagging discomfort, the outdoor-vs-indoor thermal delta tells you whether the AC is actually doing its job. The firmware classifies this directly: a delta above 8°C reads as Excellent, above 5°C as Good, above 3°C as Fair, and anything below that as Poor, signaling the AC may need maintenance.

Comfort Thresholds

The same indoor readings drive the comfort classification shown on the dashboard: below 27°C and 60% humidity reads Comfortable, below 30°C and 70% humidity reads Warning, and beyond that reads Critical. The same 70% humidity crossing is what triggers the buzzer.

Energy Tracking and Sustainability

Every second the LED stays on is logged and converted directly into energy consumed, then priced at India's standard residential and commercial rate of ₹6.5/kWh. Because the light only switches on when the room is genuinely occupied and dim, total on-time, and therefore cost, stays proportional to actual need rather than running on a fixed schedule. It is a small but direct way the system contributes toward sustainable, demand-based energy use rather than blanket illumination.

Conclusion

ComfortSense began as a hackathon prototype, but the problem it solves is not confined to one room. The same logic applies to classrooms, hospital waiting areas, and any shared space where physical comfort quietly shapes how people perform. Built around a handful of inexpensive, easily sourced components, it makes a simple case: comfort should not be an afterthought in the spaces that matter most.

About the Author

Alexander Samuel R is a second-year B.E. Electrical and Electronics Engineering student at Sri Ramakrishna Engineering College, Coimbatore.